BÀI TẬP CSP VÀ GAME TRONG AI

#. Phân tích đề bài

Mục	Nội dung
Mục tiêu	Giải 8 bài: Hill Climbing (4-Queens); Alpha-Beta + Iterative Deepening; Alpha-Beta trên cây 3 tầng; Expectimax + cắt tỉa theo cận; câu hỏi lý thuyết CSP (FC vs AC-3, MRV/LCV); mô hình hóa & giải CSP cho điều phối đường băng và Magic Square 3×3; so sánh MRV/LCV với heuristic đảo ngược.
Dữ liệu vào / ra	Đề dạng tự luận: điền chỗ trống, đánh dấu ô chọn, vẽ đồ thị. Không yêu cầu nộp code và không giới hạn thư viện. Cần ghi **Họ tên, MSSV/Lớp** ở đầu bài (để trống ở trên cho bạn tự điền).
Quy tắc nộp, chấm	Đề **không nêu** quy tắc đặt tên file hay thang điểm; bạn hãy đổi tên file theo quy định của lớp trước khi nộp.
Dữ liệu đọc từ hình	Bài 1: hậu ở A3, B1, B2, C2. Bài 2: f(B..M) theo bảng. Bài 3: 18 lá = 3,0,−1,1,4,6 | 10,−1,0,−2,12,−3 | −4,−2,−2,10,−5,−7. Bài 4: xác suất (0.2,0.5,0.3), (0.5,0.1,0.4), (0.2,0.4,0.4); lá (15,12),(8,20),(−14,15) | (−7,−2),(−18,−20),(0,15) | (18,−9),(20,−20),(−18,−10). Bạn nên đối chiếu lại với bản gốc.
Cách trình bày	Mỗi bài: hướng giải → các bước tính → **đáp án** (khung xanh). Toàn bộ kết quả đã được kiểm chứng lại bằng chương trình Python (mô phỏng thuật toán / vét cạn).

Bài 1. Bài toán 4-Queens và Hill Climbing

Hướng giải. Ký hiệu ô theo (hàng A–D từ trên xuống, cột 1–4). Từ Hình 1, bốn quân hậu đang ở **A3, B1, B2, C2**. Hàm mục tiêu h(s) là *số cặp hậu đang tấn công nhau* (cùng hàng, cột hoặc đường chéo, không bị quân khác chắn). Hill Climbing luôn nhảy tới láng giềng có h nhỏ nhất và dừng khi h $\geq$ h .

neighbor current

Tính h ban đầu: các cặp xung đột là (A3,B2) chéo, (B1,B2) cùng hàng B, (B1,C2) chéo, (B2,C2) cùng cột

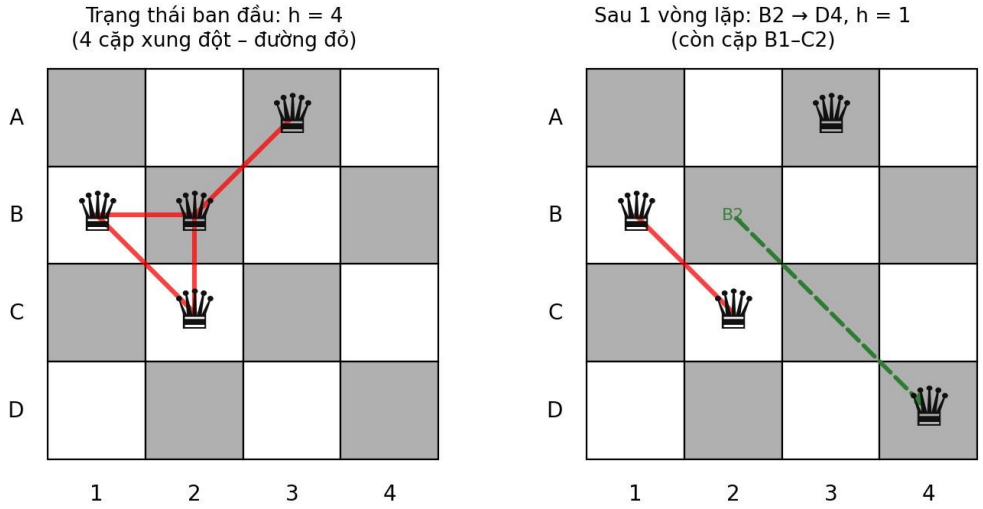
2. Các cặp (A3,B1) và (A3,C2) không thẳng hàng. Vậy **h = 4**.

a) Trạng thái sau một vòng lặp

Với mỗi quân, liệt kê mọi nước đi hợp lệ (đi thẳng/chéo, dừng trước quân chắn, không đè lên quân khác) rồi tính h của trạng thái mới:

Quân di chuyển	Số nước đi	h nhỏ nhất đạt được	Nước đi tốt nhất

Q1 (A3)	7	4	A4, B4 hoặc D3 (h = 4, không cải thiện)
Q2 (B1)	4	3	D1
Q3 (B2)	7	**1**	**D4** (đi chéo B2→C3→D4, đường thông)
Q4 (C2)	8	2	C4



Hình 1a. Trạng thái ban đầu và trạng thái sau một vòng lặp (đường đỏ = cặp xung đột).

Đáp án (a): láng giềng tốt nhất là duy nhất: di chuyển Q3 từ B2 sang D4. Sau một vòng lặp: **Q1 = A3, Q2 = B1, Q3 = D4, Q4 = C2**; h giảm từ 4 xuống 1 (còn lại cặp B1–C2 trên đường chéo).

b) Thuật toán có dừng không? Loại trạng thái cuối?

Ở vòng lặp thứ 2, trong 35 láng giềng của (A3,B1,D4,C2) tốt nhất cũng chỉ có h = 1 (ví dụ Q4: C2→C4 hoặc C2→D2). Vì h = 1 ≥ h = 1 nên điều kiện dừng thỏa và thuật toán **trả về** trạng thái hiện tại neighbor current sau đúng 1 lần di chuyển.

Phương án	**Chọn?**	**Lý do**
A) Không dừng	☐	Thuật toán dừng (điều kiện ≥ thỏa).
B) Cực trị cục bộ	☒	Không có láng giềng nào *tốt hơn hẳn* (h < 1) nên hill climbing bị kẹt tại đây.
C) Cực trị toàn cục	☐	h = 1 > 0, trong khi tồn tại nghiệm h = 0 (ví dụ A3, B1, C4, D2).
D) Vai bằng (shoulder)	☒	Có 2 láng giềng bằng h = 1, tức đang ở một cao nguyên (plateau) mà từ đó còn lối đi xuống: C2→C4 rồi D4→D2 cho h = 0. Thuật toán dừng vì không cho phép đi ngang.
E) Không thuộc trường hợp trên	☐	Đã thuộc B và D.

Đáp án (b): Có dừng. Trạng thái cuối **A3, B1, D4, C2** (h = 1) thuộc loại **B (cực trị cục bộ)** và **D (vai bằng)**; không phải A, C, E.

Ghi chú: nếu định nghĩa "cực trị cục bộ chặt" (mọi láng giềng đều xấu hơn hẳn) thì trạng thái này là cao nguyên; vì thuật toán dừng ở đó nên với góc nhìn "bị kẹt" vẫn tính là cực trị cục bộ. Hai phương án B và D cùng được chọn.

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Hướng giải. Bước 1: sắp thứ tự con của A (nút MAX) theo f giảm dần (tốt nhất cho MAX trước) để cắt tỉa hiệu quả. Bước 2: chạy Alpha-Beta tới các nút cháu và dùng f tại nút cháu. Cắt tỉa tại nút MIN khi $v \le \alpha$. **a) Thứ tự các cây con**

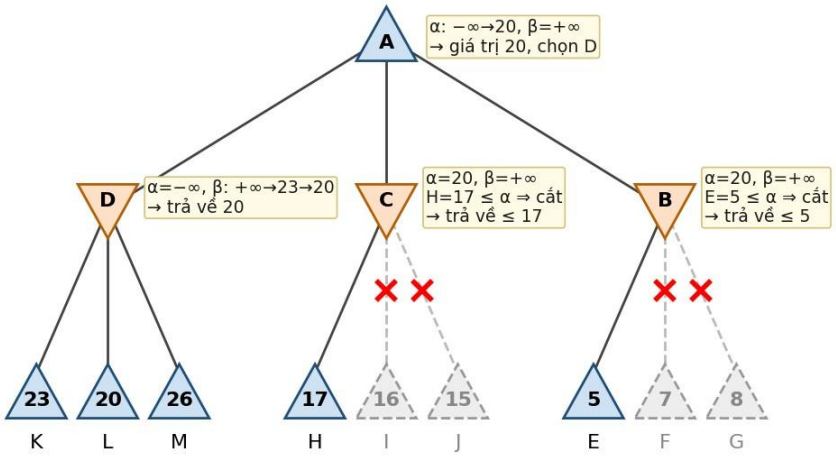
f(B) = 4, f(C) = 14, f(D) = 19. Theo góc nhìn nút cha MAX, giá trị càng lớn càng tốt, nên thứ tự là D (19) > C (14) > B (4).

Phương án	Chọn?
B → C → D	☐
C → D → B	☐
D → C → B	☒

Đáp án (a): D → C → B.

b) Các nút không được thăm do bị cắt tỉa

Nút	α vào	β vào	Diễn biến	Trả về
A (MAX)	$-\infty$	$+\infty$	α: $-\infty$ → 20 sau khi xong D	20
D (MIN)	$-\infty$	$+\infty$	K = 23 → β = 23; L = 20 → β = 20; M = 26 không đổi	20
C (MIN)	20	$+\infty$	H = 17 ≤ α = 20 ⇒ **cắt I, J**	≤ 17
B (MIN)	20	$+\infty$	E = 5 ≤ α = 20 ⇒ **cắt F, G**	≤ 5



Hình 2a. Alpha-Beta với thứ tự D → C → B (✕ = nhánh bị cắt).

Đáp án (b): các nút không được thăm: **I, J** (dưới C) và **F, G** (dưới B). Chỉ thăm 5/9 nút cháu (K, L, M, H, E). Giá trị tại A = 20, nước đi tốt nhất là D.

So sánh: với thứ tự gốc B → C → D thì α luôn nhỏ (5, rồi 15) nên không nút nào bị cắt (thăm đủ 9/9 nút cháu). Sắp xếp theo f giúp cắt thêm 4 nút.

c) Iterative deepening + pruning có luôn cho cùng nước đi với Minimax đầy đủ?

Phương án	Chọn?
Có	☐
Không	☒

Giải thích. Alpha-Beta (dù đổi thứ tự con thế nào) luôn trả về đúng giá trị Minimax *của cây đang được tìm kiếm*; thứ tự chỉ ảnh hưởng số nút bị cắt. Tuy nhiên ở đây cây tìm kiếm đã bị **cắt độ sâu** (dừng ở nút cháu) và dùng **hàm đánh giá f(s)**, chỉ là ước lượng heuristic của giá trị thật. Giá trị trên cây rút gọn có thể khác Minimax trên toàn bộ cây (sai số của f, hiệu ứng chân trời), nên nước đi có thể khác. Chỉ khi tìm tới các trạng thái kết thúc (hoặc f chính xác) thì mới đảm bảo trùng với Minimax đầy đủ. Lưu ý thêm: các nút bị cắt trả về cận (≤ 17, ≤ 5), không phải giá trị đúng, nhưng điều đó không làm sai giá trị ở gốc.

Đáp án (c): Không – vì tìm kiếm giới hạn độ sâu và dựa vào hàm đánh giá heuristic; chỉ đảm bảo đúng Minimax của cây đã bị cắt độ sâu, không phải của toàn bộ cây trò chơi.

Bài 3. Adversarial Search – Alpha-Beta Pruning

Hướng giải. Cây có 3 tầng: gốc MAX → 3 nút MIN → 9 nút MAX (mỗi nút 2 lá). Duyệt trái sang phải, α khởi tạo −∞, β khởi tạo +∞. Tại nút MAX: cập nhật v = max, **cắt nếu v ≥ β**, sau đó α = max(α, v). Tại nút MIN: v = min, **cắt nếu v ≤ α**, sau đó β = min(β, v).

Giá trị Minimax đầy đủ (để đối chiếu): các nút MAX tầng 2 = 3, 1, 6 | 10, 0, 12 | −2, 10, −5; ba nút MIN = 1, 0, −5; gốc = max(1, 0, −5) = **1**.

Nút	α vào	β vào	Diễn biến	Trả về
Gốc (MAX)	−∞	+∞	α: −∞ → 1 sau nhánh trái; nhánh giữa, phải không làm đổi	1
MIN 1	−∞	+∞	β: +∞ → 3 → 1	1
MAX 1.1	−∞	+∞	lá 3 → v = 3, α = 3; lá 0 → v = 3	3
MAX 1.2	−∞	3	lá −1 → α = −1; lá 1 → v = 1, α = 1 (1 < β = 3, không cắt)	1
MAX 1.3	−∞	1	lá 4 → v = 4 ≥ β = 1 ⇒ **cắt lá 6**	4
MIN 2	1	+∞	β: +∞ → 10 → 0; v = 0 ≤ α = 1 ⇒ **cắt MAX 2.3 (lá 12, −3)**	0

MAX 2.1	1	+∞	lá 10 → v = 10, α = 10; lá −1 → v = 10	10
MAX 2.2	1	10	lá 0 → v = 0; lá −2 → v = 0 (α giữ 1)	0
MIN 3	1	+∞	β: +∞ → −2; v = −2 ≤ α = 1 ⇒ **cắt MAX 3.2 và MAX 3.3**	−2
MAX 3.1	1	+∞	lá −4 → v = −4; lá −2 → v = −2	−2

Đáp án: Giá trị Minimax tại gốc = **1**; nước đi tối ưu của MAX là **nhánh trái** (nút MIN thứ nhất). Các nhánh bị cắt (✕): lá 6 (dưới MAX 1.3); MAX 2.3 cùng lá 12, −3; MAX 3.2 (lá −2, 10) và MAX 3.3 (lá −5, −7). Tổng cộng thăm 11/18 lá, cắt 7 lá. Hình vẽ ở trang kế tiếp (Hình 3).

Bài 4. Expectimax và Chance Nodes

Hướng giải. Nút MAX = max các lá; chance node = tổng có trọng số E = Σ p·v; gốc = max các chance i i node.

a) Giá trị tại từng nút

Chance node (xác suất)	MAX con 1	MAX con 2	MAX con 3	Giá trị chance node
C1 (0.2 / 0.5 / 0.3)	max(15, 12) = 15	max(8, 20) = 20	max(−14, 15) = 15	0.2·15 + 0.5·20 + 0.3·15 = 3 + 10 + 4.5 = **17.5**
C2 (0.5 / 0.1 / 0.4)	max(−7, −2) = −2	max(−18, −20) = −18	max(0, 15) = 15	0.5·(−2) + 0.1·(−18) + 0.4·15 = −1 − 1.8 + 6 = **3.2**
C3 (0.2 / 0.4 / 0.4)	max(18, −9) = 18	max(20, −20) = 20	max(−18, −10) = −10	0.2·18 + 0.4·20 + 0.4·(−10) = 3.6 + 8 − 4 = **7.6**

Đáp án (a): các nút MAX (trái→phải) = 15, 20, 15 | −2, −18, 15 | 18, 20, −10; ba chance node = 17.5, 3.2, 7.6; **nút gốc = max(17.5, 3.2, 7.6) = 17.5** (chọn nhánh trái). Xác suất mỗi chance node đều cộng bằng 1. Hình 4a ở trang kế tiếp.

b) Cắt tỉa khi chưa biết giới hạn giá trị lá

Không nút nào có thể bị cắt. Lý do: (1) giá trị chance node là *kỳ vọng* – một tổng có trọng số, phụ thuộc vào giá trị cụ thể chứ không chỉ thứ tự so sánh như min/max; (2) mỗi nhánh có xác suất > 0, nên một lá chưa xem có thể lớn hoặc nhỏ tùy ý và đẩy kỳ vọng lên/xuống bất kỳ mức nào; (3) tại nút MAX bên dưới, lá chưa xem cũng có thể lớn hơn lá đã thấy; (4) ở gốc, muốn bỏ một chance node cần một *cận trên* của nó, mà không có cận trên của lá thì không có cận trên của kỳ vọng. Do đó mọi nút phải được tính để kết quả đúng.

Đáp án (b): không cắt được nút nào (cả 18 lá phải được xét).

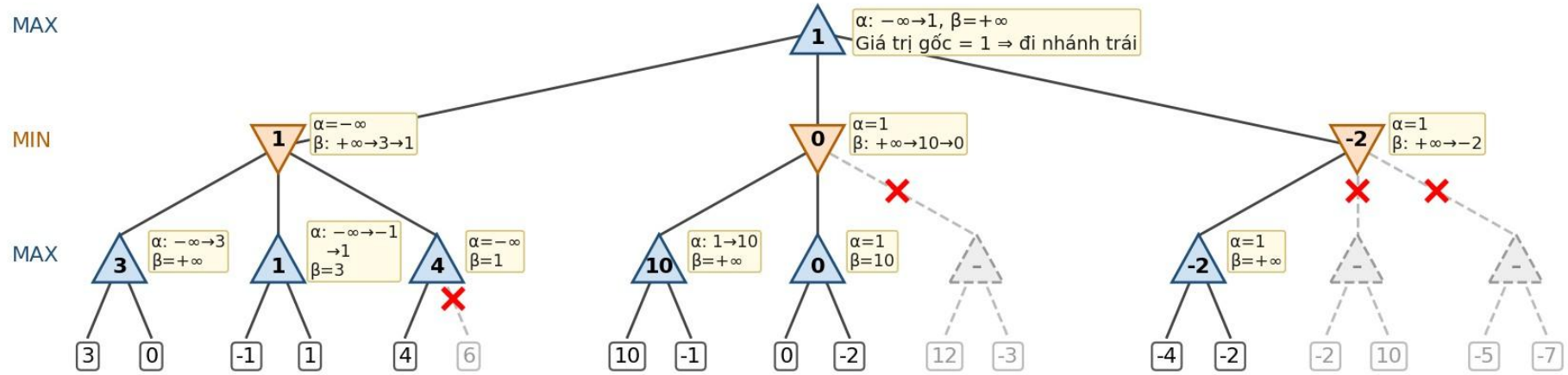
c) Lá nằm trong [−20, 20]

Với cận trên U = 20, tại chance node đã tính một phần: **U = Σ p ·v (các con đã biết) + (xác suất còn** lá **i i lại)·20**. Nếu U ≤ giá trị tốt nhất đã có ở gốc (best) thì cắt các con còn lại. Tại nút MAX: nếu giá trị đạt 20 thì cắt các lá còn lại.

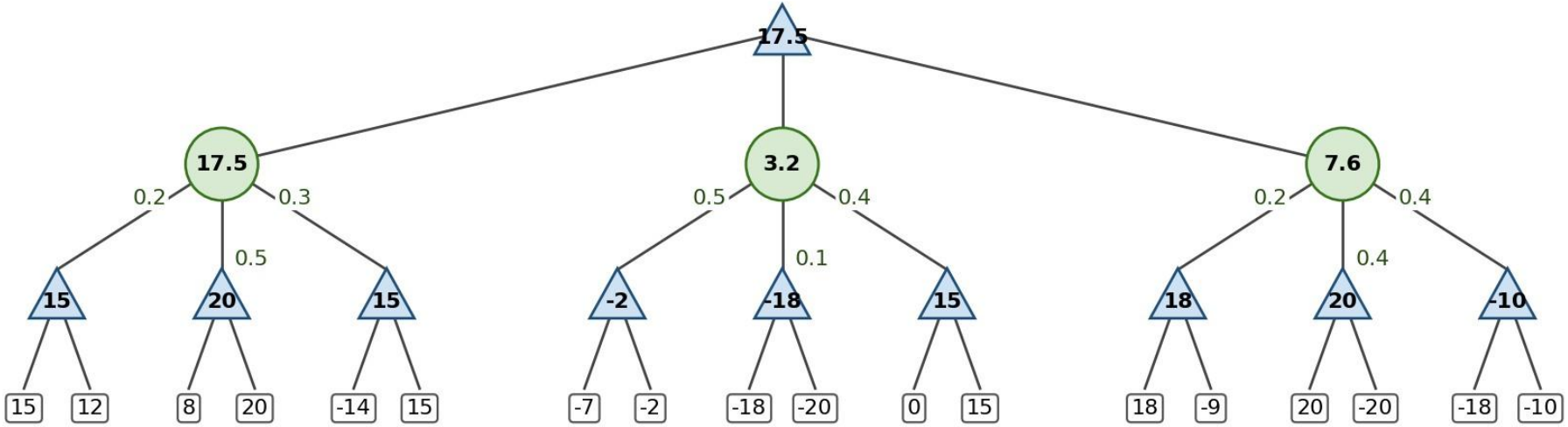
Bước	Duyệt	Kết quả / điều kiện cắt
1. C1	15,12 → 15; 8,20 → 20; −14,15 → 15	Chưa có best, các MAX không chạm 20 trước lá cuối ⇒ không cắt. C1 = 17.5, best = 17.5.
2. C2	−7, −2 → MAX = −2	U = 0.5·(−2) + (0.1+0.4)·20 = −1 + 10 = **9 ≤ 17.5 ⇒ cắt MAX thứ 2 và 3** (lá −18, −20, 0, 15). C2 trả về cận ≤ 9.
3. C3	18, −9 → 18	U = 3.6 + 0.8·20 = 19.6 > 17.5, tiếp tục.
	20 → MAX = 20 (= cận trên)	**Cắt lá −20**. U = 3.6 + 8 + 0.4·20 = 19.6 > 17.5, vẫn chưa cắt được con thứ 3.
	−18, −10 → −10	C3 = 3.6 + 8 − 4 = 7.6 (chính xác).

Đáp án (c): gốc = **17.5**, chọn nhánh trái. Nhánh bị cắt (✕): hai nút MAX cuối của C2 (4 lá: −18, −20, 0, 15) và lá −20 dưới nút MAX giữa của C3. Tổng cộng cắt 5/18 lá (thăm 13 lá). Hình 4c ở trang kế tiếp.

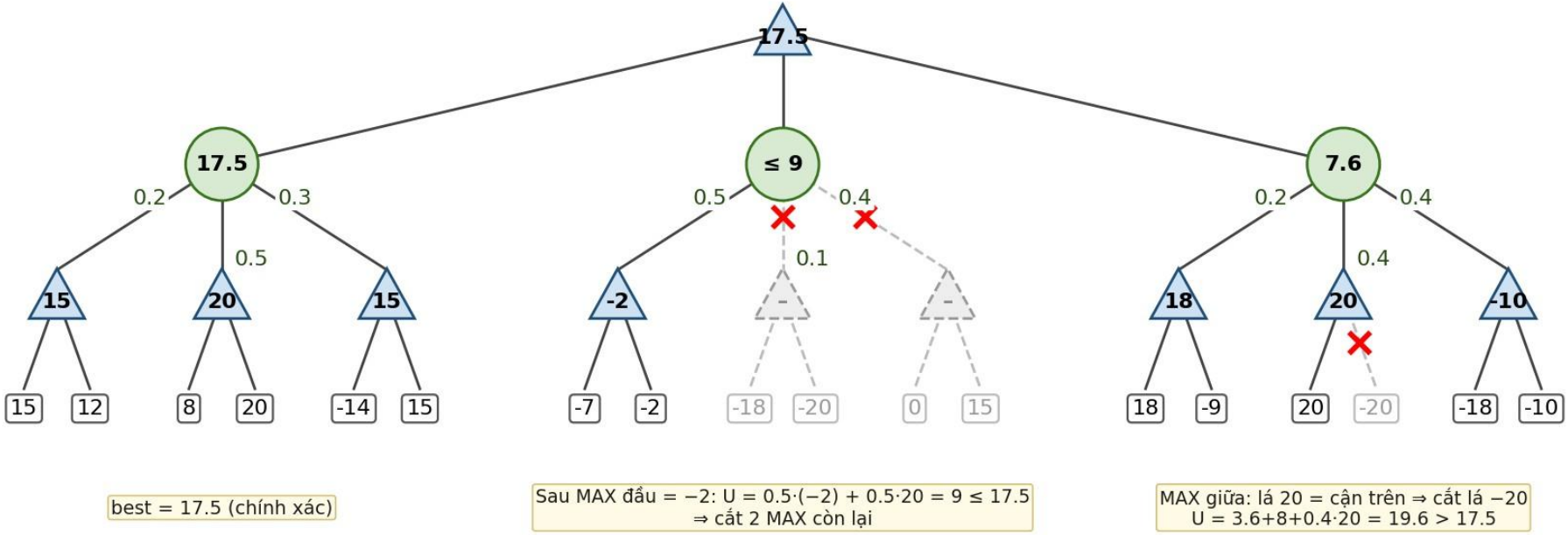
Hình lời giải Bài 3 và Bài 4



Hình 3. Alpha-Beta Pruning: α, β tại mỗi nút khi duyệt (hộp vàng), giá trị trả về ghi trong nút, ✕ = nhánh bị cắt, nút/lá xám = không được thăm.



Hình 4a. Expectimax (không cắt tỉa): giá trị tại mỗi nút MAX, chance node và gốc.



Hình 4c. Expectimax với lá $\in [-20, 20]$: ✕ = nhánh bị cắt; chance node giữa chỉ trả về cận trên ≤ 9.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào?

Tiêu chí	Forward Checking (FC)	AC-3
Phạm vi lan truyền	Chỉ **một bước**: sau khi gán X = v, loại khỏi miền của các biến *chưa gán kề X* những giá trị xung đột với v. Không lan truyền tiếp (nếu miền của Y bị thu hẹp, không kiểm tra hệ quả lên Z).	**Lan truyền chuỗi**: khi miền của Y thu hẹp, đưa các cung (Z, Y) vào hàng đợi kiểm tra lại, lặp tới khi mọi cung nhất quán; áp dụng cho cả cặp biến chưa gán với nhau.
Mức nhất quán	Tương đương nhất quán cung chỉ trên các cung hướng tới biến vừa gán ⇒ yếu hơn.	Nhất quán cung toàn bộ đồ thị ràng buộc ⇒ mạnh hơn, phát hiện sớm hơn miền rỗng.
Thời điểm thực hiện	Trong quá trình tìm kiếm quay lui, sau **mỗi phép gán** (và hoàn tác khi quay lui).	Có thể làm **tiền xử lý** trước khi tìm kiếm, hoặc gọi lại sau mỗi phép gán (MAC – Maintaining Arc Consistency).
Chi phí	Rẻ: khoảng O(d·deg) mỗi phép gán (d = cỡ miền, deg = số láng giềng).	Đắt hơn: O(e·d^3) trong trường hợp xấu nhất (e = số cung), nhưng có thể giảm mạnh không gian tìm kiếm.

Minh họa (Bài 6d vs 6c): sau B = 1, A = 2, FC chỉ còn C, D ∈ {3, 4} và phải thử C = 3 mới phát hiện miền D rỗng; AC-3 suy ra ngay C = {4}, D = {3}.

> **Đáp án (a):** FC chỉ lọc miền của láng giềng chưa gán của biến vừa gán (1 bước, rẻ, chạy trong lúc search); AC-3 lan truyền cung tới khi ổn định (mạnh hơn, đắt hơn, dùng tiền xử lý hoặc sau mỗi phép gán).

b) Vì sao dùng các heuristic sau?

i) **MRV (Minimum Remaining Values)** – chọn biến có ít giá trị hợp lệ nhất (nguyên tắc *fail-first*): (1) biến bị ràng buộc chặt nhất dễ dẫn tới thất bại nhất, nên phát hiện ngõ cụt sớm, ở tầng nông của cây, cắt được cả nhánh lớn; (2) biến chỉ còn 1 giá trị là nước đi bắt buộc, gán sớm giúp lan truyền ràng buộc; (3) giảm hệ số nhánh hiệu dụng. Vì mọi biến cuối cùng đều phải được gán, thứ tự biến chỉ ảnh hưởng độ lớn cây tìm kiếm chứ không ảnh hưởng tính đúng.

ii) **LCV (Least Constraining Value)** – với biến đã chọn, thử trước giá trị làm mất ít giá trị nhất trong miền các biến láng giềng: giữ lại nhiều lựa chọn cho các biến sau, nên khả năng tìm được nghiệm mà không phải quay lui cao hơn (nguyên tắc *succeed-first*). Khác với biến, ta chỉ cần *một* giá trị đúng của biến hiện tại, nên chọn giá trị "dễ thành công nhất" trước. LCV không đổi tính đúng, chỉ giảm số lần quay lui khi bài toán có nghiệm.

> **Đáp án (b):** MRV = chọn biến khó nhất trước để thất bại sớm và thu nhỏ cây; LCV = chọn giá trị ít ràng buộc nhất để tăng khả năng thành công và giảm quay lui.

Bài 6. CSP – Điều phối không lưu

a) Mô hình hóa CSP

Ký hiệu QT = đường băng quốc tế, ND = đường băng nội địa; mỗi máy bay X có khung giờ t và đường

X

băng r . X

Thành phần	Mô hình
Biến	X = {A, B, C, D, E}; mỗi biến X = (t , r) X X
Miền giá trị	D = {1, 2, 3, 4} × {QT, ND} (8 giá trị) với mọi X X
Ràng buộc đơn (unary)	B: t = 1 | D: t ≥ 3 | A: t ≤ 2 B D A
Ràng buộc nhị phân (binary)	D hạ cánh trước khi C cất cánh: t < t (giả sử nghiêm ngặt) D C Không hai máy bay cùng khung giờ trên cùng đường băng: ∀ X ≠ Y: ¬(t = t ∧ r = r), X Y X Y tức (t , r) ≠ (t , r) (10 cặp) X X Y Y

Sau khi áp dụng unary: D = {1,2}×{QT,ND}; D = {(1,QT),(1,ND)}; D = {3,4}×{QT,ND}; C, E giữ nguyên 8 giá trị.

A B D

b) Thêm ràng buộc loại đường băng: miền mới và đồ thị ràng buộc

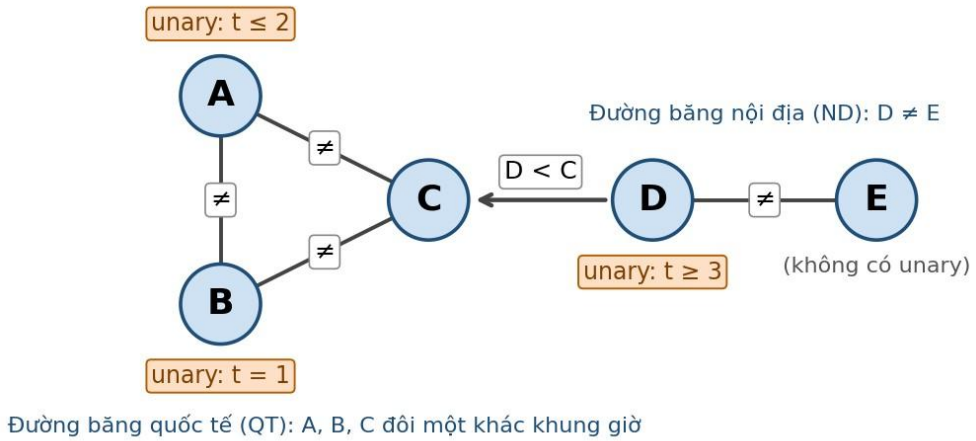
A, B, C ∈ QT; D, E ∈ ND nên r đã cố định; miền của mỗi biến chỉ còn khung giờ: **D = {1, 2, 3, 4}**. Ràng

X **X**

buộc "khác ô" chỉ còn giữa các máy bay *cùng đường băng*: A, B, C (đôi một khác giờ) và D, E (khác giờ).

Hai máy bay khác đường băng không ràng buộc nhau về ô, nên không có cạnh.

Loại	Ràng buộc
Unary	t = 1; t ≥ 3; t ≤ 2 B D A
Binary	t ≠ t , t ≠ t , t ≠ t ; t ≠ t ; t < t A B A C B C D E D C



Hình 6b. Đồ thị ràng buộc: tam giác A–B–C (≠), cung D → C (D < C) và cạnh D–E (≠).

c) Arc consistency (bắt đầu từ unary)

Sau unary: A {1,2}, B {1}, C {1,2,3,4}, D {3,4}, E {1,2,3,4}. Chạy AC-3:

Bước	Cung xét	Giá trị bị loại	Miền mới
1	(A, B): A ≠ B, B = 1	A: 1	A = {2}
2	(C, B): C ≠ B	C: 1	C = {2,3,4}
3	(C, A): C ≠ A, A = 2	C: 2	C = {3,4}
4	(D, C): D < C	D: 4 (cần C > 4, không có)	D = {3}
5	(C, D): D < C, D = 3	C: 3 (cần D < 3, không có)	C = {4}
6	(E, D): E ≠ D, D = 3	E: 3	E = {1,2,4}
7	Kiểm tra lại (A,C), (B,C), (D,E)…	không còn thay đổi	hội tụ

Biến	1	2	3	4	Miền còn lại
A	~~1~~(1)	2	~~3~~u	~~4~~u	{2}
B	1	~~2~~u	~~3~~u	~~4~~u	{1}
C	~~1~~(2)	~~2~~(3)	~~3~~(5)	4	{4}
D	~~1~~u	~~2~~u	3	~~4~~(4)	{3}
E	1	2	~~3~~(6)	4	{1, 2, 4}

Chú thích: u = bị loại bởi ràng buộc unary; (k) = bị loại ở bước k của AC-3.

Đáp án (c): A = {2}, B = {1}, C = {4}, D = {3}, E = {1, 2, 4}. Bốn máy bay A, B, C, D đã xác định hoàn toàn (A: ô 2, B: ô 1, C: ô 4, D: ô 3); E có thể ở ô 1, 2 hoặc 4 của đường băng nội địa.

d) Forward Checking + Backtracking (MRV; hòa → ký tự nhỏ trước; giá trị nhỏ trước)

Miền sau unary: A {1,2}, B {1}, C {1,2,3,4}, D {3,4}, E {1,2,3,4}.

#	Biến chọn (MRV)	Phép gán	Forward Checking → miền còn lại	Ghi chú

1	B (|D| = 1)	**(B, 1)**	A {2}; C {2,3,4}	
2	A (|D| = 1)	**(A, 2)**	C {3,4}	
#	**Biến chọn (MRV)**	**Phép gán**	**Forward Checking → miền còn lại**	**Ghi chú**
3	C và D cùng 2 giá trị ⇒ chọn C (chữ nhỏ hơn)	**(C, 3)**	D < 3 ⇒ D = ∅	**Dead end** ⇒ hoàn tác (C, 3)
4	C	**(C, 4)**	D {3}	
5	D (|D| = 1)	**(D, 3)**	E {1,2,4}	
6	E	**(E, 1)**	–	Đủ 5 biến ⇒ nghiệm

Đáp án (d): thứ tự các cặp (biến, giá trị): **(B,1), (A,2), (C,3) ✗ hoàn tác, (C,4), (D,3), (E,1)**. Nghiệm: A = 2, B = 1, C = 4, D = 3, E = 1, tức A, B, C trên QT ở ô 2, 1, 4 và D, E trên ND ở ô 3, 1. Có 1 lần quay lui (tại C = 3).

Bài 7. CSP – Magic Square

Bảng (chữ cái là ô cần tìm): hàng 1 = 4, A, 2 | hàng 2 = B, 5, C | hàng 3 = D, E, 6; tổng mỗi hàng/cột/đường chéo = 15.

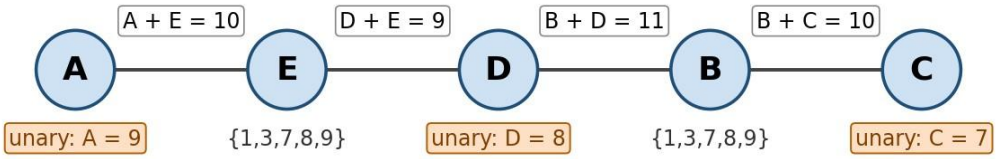
a) Mô hình CSP

Thành phần	Mô hình
Biến	A, B, C, D, E
Miền	Ban đầu {1,…,9}. Do 4, 2, 5, 6 đã dùng và các ô phải khác nhau nên loại 2, 4, 5, 6 ⇒ D = … A = D = {1, 3, 7, 8, 9} E
Ràng buộc all-diff	alldiff(A, B, C, D, E) (và khác 2, 4, 5, 6)
Hàng 1	4 + A + 2 = 15 ⇒ **A = 9** (unary)
Hàng 2	B + 5 + C = 15 ⇒ **B + C = 10** (binary)
Hàng 3	D + E + 6 = 15 ⇒ **D + E = 9** (binary)
Cột 1	4 + B + D = 15 ⇒ **B + D = 11** (binary)
Cột 2	A + 5 + E = 15 ⇒ **A + E = 10** (binary)
Cột 3	2 + C + 6 = 15 ⇒ **C = 7** (unary)
Đường chéo chính	4 + 5 + 6 = 15: luôn đúng (không có biến)

Đường chéo phụ	2 + 5 + D = 15 ⇒ **D = 8** (unary)

b) Đồ thị ràng buộc nhị phân (bỏ alldiff)

Các ràng buộc nhị phân: B+C = 10, D+E = 9, B+D = 11, A+E = 10 ⇒ đồ thị là một **chuỗi (cây)**: A — E — D — B — C. Các ràng buộc unary A = 9, C = 7, D = 8 gắn lên nút tương ứng.



Hình 7b. Đồ thị ràng buộc nhị phân của Magic Square 3×3.

c) AC-3 và tìm nghiệm

Sau unary: A {9}, C {7}, D {8}, B và E {1,3,7,8,9}.

Bước	Cung xét	Giá trị bị loại	Miền mới
1	(B, C): B + C = 10, C = 7	B ∈ {1,7,8,9} (cần C = 9, 3, 2, 1: không có)	B = {3}
2	(E, D): D + E = 9, D = 8	E ∈ {3,7,8,9} (cần D = 6, 2, 1, 0)	E = {1}
3	(D, B), (B, D): B + D = 11	không loại (3 + 8 = 11 ✓)	giữ
4	(A, E), (E, A): A + E = 10	không loại (9 + 1 = 10 ✓)	giữ
Bước	**Cung xét**	**Giá trị bị loại**	**Miền mới**
5	Cung (C,B), (D,E) được đưa lại hàng đợi	không loại	hội tụ

Đáp án (c): sau AC-3 mọi miền chỉ còn một giá trị ⇒ **A = 9, B = 3, C = 7, D = 8, E = 1** (không cần quay lui; đồ thị dạng cây nên AC là đủ). Kiểm tra: hàng 4+9+2 = 3+5+7 = 8+1+6 = 15; cột 4+3+8 = 9+5+1 = 2+7+6 = 15; chéo 4+5+6 = 2+5+8 = 15; chín số {4,9,2,3,5,7,8,1,6} đôi một khác nhau.

4	**9**	**2**
3	**5**	**7**
8	**1**	**6**

Magic Square hoàn chỉnh (số xanh là các ô cần tìm).

Bài 8. MRV và LCV trong thực hành

Miền: AJ = {Đỏ, Vàng, Xanh lá}; Sunny = {Đỏ, Vàng, Hồng}; Ellyse = {Đỏ, Hồng}; ba người phải chọn ba màu khác nhau (đồ thị đầy đủ K). Thứ tự cầu vồng: Đỏ < Vàng < Xanh lá (Hồng không thuộc cầu vồng,

xếp sau cùng). Dùng Forward Checking để cập nhật miền.

a) MRV + LCV

Bước	Chọn biến (MRV)	Chọn giá trị (LCV: ít giá trị bị loại nhất ở láng giềng)	Kết quả
1	Ellyse (2 giá trị, ít nhất)	Đỏ: loại Đỏ khỏi AJ và Sunny = 2 giá trị Hồng: chỉ loại Hồng khỏi Sunny = **1** ⇒ chọn **Hồng**	Ellyse = Hồng; Sunny {Đỏ, Vàng}; AJ {Đỏ, Vàng, Xanh lá}
2	Sunny (2) < AJ (3)	Đỏ: loại Đỏ khỏi AJ = 1; Vàng: loại Vàng khỏi AJ = 1; hòa ⇒ cầu vồng: **Đỏ**	Sunny = Đỏ; AJ {Vàng, Xanh lá}
3	AJ	Cả hai đều loại 0; hòa ⇒ **Vàng**	AJ = Vàng

Đáp án (a): thứ tự gán: **Ellyse = Hồng → Sunny = Đỏ → AJ = Vàng**. Số lần backtrack = **0**.

b) Maximum Remaining Values + Most Constraining Value

Bước	Chọn biến (miền lớn nhất)	Chọn giá trị (loại nhiều nhất)	Kết quả
1	AJ và Sunny cùng 3 giá trị; hòa ⇒ theo thứ tự đề (AJ trước)	Đỏ: loại 2 (Sunny, Ellyse); Vàng: loại 1; Xanh lá: loại 0 ⇒ **Đỏ**	AJ = Đỏ; Sunny {Vàng, Hồng}; Ellyse {Hồng}
2	Sunny (2) > Ellyse (1)	Hồng: loại Hồng của Ellyse = 1; Vàng: loại 0 ⇒ **Hồng**	Sunny = Hồng ⇒ Ellyse = ∅ **dead end**
3	Quay lui về Sunny	Giá trị kế tiếp: Vàng	Sunny = Vàng; Ellyse {Hồng}
4	Ellyse	Hồng	Ellyse = Hồng ⇒ nghiệm

Đáp án (b): thứ tự gán: AJ = Đỏ → Sunny = Hồng ✗ → (backtrack) Sunny = Vàng → Ellyse = Hồng. Số lần backtrack = **1**, nhiều hơn câu (a) (0 lần).

Giải thích. Heuristic đảo ngược làm ngược nguyên tắc: chọn biến lỏng nhất (AJ) nên ngõ cụt của Ellyse (chỉ có 2 màu) bị đẩy xuống muộn; chọn giá trị phá hoại nhất (Đỏ, Hồng) làm co miền láng giềng nhanh nhất, trực tiếp dẫn tới miền rỗng. Ngược lại, MRV xử lý ngay biến khó nhất (Ellyse) và LCV chọn Hồng (chỉ ảnh hưởng 1 giá trị), giữ cho Sunny và AJ còn nhiều lựa chọn nên không phải quay lui.

Lưu ý: số lần backtrack ở (b) phụ thuộc cách phá hòa biến; ở đây dùng thứ tự liệt kê trong đề (AJ → Sunny → Ellyse). Với bài rất nhỏ như thế này kết quả có thể thay đổi theo cách phá hòa, nhưng về tổng quát MRV/LCV tốt hơn bản đảo ngược.

Tổng hợp đáp án

Bài	Đáp án chính
1a	Q1 = A3, Q2 = B1, Q3 = D4 (B2 → D4), Q4 = C2; h: 4 → 1

Bài	Đáp án chính
1b	Dừng; chọn B (cực trị cục bộ) và D (vai bằng); không phải A, C, E
2	a) D → C → B b) cắt I, J, F, G c) Không (cắt độ sâu + hàm đánh giá heuristic)
3	Gốc = 1, đi nhánh trái; cắt lá 6; MAX 2.3 (12, −3); MAX 3.2 (−2, 10); MAX 3.3 (−5, −7)
4	a) MAX: 15,20,15 | −2,−18,15 | 18,20,−10; chance: 17.5, 3.2, 7.6; gốc 17.5 b) không cắt được c) cắt 2 MAX cuối của C2 và lá −20 của C3; gốc 17.5
5	a) FC: 1 bước, rẻ, trong search; AC-3: lan truyền tới ổn định, đắt hơn, mạnh hơn b) MRV: fail-first; LCV: để lại nhiều lựa chọn
6	b) miền {1,2,3,4}; đồ thị: A–B–C (≠), D→C (<), D–E (≠) c) A{2}, B{1}, C{4}, D{3}, E{1,2,4} d) (B,1),(A,2),(C,3)✗,(C,4),(D,3),(E,1)
7	A = 9, B = 3, C = 7, D = 8, E = 1; đồ thị chuỗi A–E–D–B–C
8	a) Hồng → Đỏ → Vàng (Ellyse, Sunny, AJ), 0 backtrack b) 1 backtrack